

JDCA Scoring App

Operations Runbook

Jabalpur District Cricket Association

The single reference for running, maintaining, and recovering the application **without a developer on call**. If something goes wrong, start here.

Document version	1.0
Prepared	10 October 2026
Applies to	JDCA React PWA — Vercel + Supabase + Cloudinary
Audience	Application owner / administrator (non-developer)
Review cycle	Re-check accounts & backups every 6 months

HOW TO USE THIS: Sections 1–3 are the system map and the accounts you must own. Sections 4–7 are day-to-day tasks and emergency recovery. Keep this PDF and the master credentials in a safe, shared place (e.g. a password manager vault) — not on one person's laptop.

1. What the system is made of

The app has **no server you have to run**. It is a website that talks directly to managed cloud services. That is deliberate — it means there is very little that can break on its own, and nothing that needs patching or restarting. Four providers do all the work:

Provider	What it does
Vercel	Hosts the website. Rebuilds and re-deploys automatically whenever code is pushed to the GitHub repository's main branch. Free/hobby tier is sufficient.
Supabase	The database, user login/authentication, and real-time live-score updates. This is the heart of the system — all match data lives here.
Cloudinary	Stores uploaded images (player photos, logos). Non-critical: if it is down, only images are affected, not scoring.
GitHub	Stores the source code and runs two automated jobs: a database backup every 3 days and a daily 'keep-alive' ping so Supabase does not auto-pause.

How a score reaches every viewer

- A scorer records each ball. It is saved **instantly on their device** first (works fully offline).
- A background sync worker pushes each ball to Supabase **in strict order**, one at a time.
- Supabase broadcasts the change over real-time channels, so spectators' screens update live.
- If the scorer is offline, balls queue on the device and sync automatically when the connection returns — nothing is lost.

WHY THIS MATTERS FOR YOU: Because scoring is offline-first and self-healing, the most common 'failures' are not data loss — they are a scorer who has been **quietly signed out** and doesn't realise their balls aren't uploading. Section 6 covers how to spot and fix this.

2. Accounts you must own (do this first)

If you walk away as the developer, **ownership of these accounts must transfer to the association**, ideally to a shared organisation account rather than one person. If access is lost, the app cannot be fixed, re-deployed, or recovered. This is the single most important page in this document.

Account	What level of access is required	Where
Vercel	Project owner + ability to see Environment Variables and redeploy	vercel.com
Supabase	Project owner (Organization owner) — holds the database & all data	supabase.com
GitHub	Admin of the code repository + Actions secrets	github.com
Cloudinary	Account owner (cloud name, API key/secret, upload preset)	cloudinary.com
Sentry (optional)	Member access to see production errors	sentry.io
Domain registrar	If a custom domain is used, the registrar login	(your registrar)

ACTION: Confirm each account above is owned by an association email the organisation controls — not a personal developer email. Store every login in one shared password manager vault.

3. Configuration secrets (environment variables)

These values connect the website to its services. They are **not** stored in the code (the code is public-safe). They live in two places: **Vercel** (for the live website) and **GitHub Actions secrets** (for the backup & keep-alive jobs). If you ever move hosting, these must be copied across.

Set in Vercel → Project → Settings → Environment Variables

Variable	Meaning	Required?
VITE_SUPABASE_URL	Supabase project URL	Yes — app won't load without it
VITE_SUPABASE_ANON_KEY	Supabase public API key	Yes
VITE_CLOUDINARY_CLOUD_NAME	Cloudinary cloud name	For image uploads
VITE_CLOUDINARY_API_KEY	Cloudinary API key	For image uploads
VITE_CLOUDINARY_UPLOAD_PRESET	Unsigned upload preset name	For image uploads
VITE_SENTRY_DSN	Sentry error-tracking address	Optional but recommended

Set in GitHub → Repo → Settings → Secrets and variables → Actions

Secret	Used by
SUPABASE_DB_URL	Full database connection string — used only by the backup job
VITE_SUPABASE_URL	Used by the daily keep-alive ping
VITE_SUPABASE_ANON_KEY	Used by the daily keep-alive ping

NEVER DO THIS: Never put the Cloudinary **API secret** or the Supabase **service-role key** into any VITE_ variable or front-end code — anything prefixed VITE_ is shipped to every browser. The anon key is safe to expose; the service role key is not.

4. Routine administration tasks

Add or change a user's role

Roles, from most to least access: **SUPER_ADMIN** (everything), **DISTRICT_ADMIN**, **SELECTOR**, **SCORER** (can score matches), **VIEWER** (read-only, the default).

- A new person signs up in the app → they are created as VIEWER automatically.
- In Supabase → Table Editor → **profiles** table, find their row and set the **role** column and the **can_add** / **can_edit** / **can_delete** flags as needed.

- To give a SELECTOR their age-group scope, set **selector_age_category_id**.
- Role changes take effect the next time they reload the app.

Deactivate a user (recommended over deleting)

- In the **profiles** table set **is_active = false**. They are signed out and dropped to no access on next load.
- This is reversible and keeps their history intact — prefer it to deletion.

Permanently delete a user

Only a SUPER_ADMIN can do this. It is done through the app's admin screen, which calls a secure database function (`admin_delete_user`). Avoid unless legally required — deleting removes the login and cascades to their profile.

Deploy a change / roll back

- **Deploy:** pushing to the GitHub main branch makes Vercel build and publish automatically within ~1–2 minutes.
- **Roll back:** Vercel → Deployments → pick the last known-good deployment → **Promote to Production**. This instantly reverts the live site with no code changes.
- The app auto-updates in users' browsers after a deploy — no action needed from scorers.

5. Making a database (SQL) change

IMPORTANT LIMITATION: Database changes were applied by hand through the Supabase SQL Editor, not through an automated migration system. There are many numbered .sql files in the code repository and the live database has drifted from them. **Treat the live Supabase database as the source of truth**, not the .sql files.

Safe procedure for any schema change

- **Trigger a manual backup first** (Section 6) and confirm it completed.
- Open Supabase → SQL Editor. Paste the change. Read it once more before running.
- Run it. If it touches Row-Level-Security (RLS) policies, immediately test the affected role in the app (e.g. can a SCORER still record a ball?).
- Save the exact SQL you ran into a dated note, so there is a record of what changed and when.

If a legitimate action suddenly 'fails to sync'

The most likely cause is a missing or wrong RLS policy on a table the user is writing to (the app reports these as a permanent sync failure for that action). Check the policies on the relevant table in Supabase → Authentication → Policies before assuming the data is corrupt.

6. Backups and disaster recovery

How backups work today

- A GitHub Action (**Automated Database Backup**) runs **every 3 days** and on-demand. It dumps the roles, schema, and data of the Supabase database.

- The backup is stored as a **GitHub Actions artifact** and kept for **90 days**.
- A second Action pings Supabase daily so the free-tier project is not auto-paused for inactivity.

SURVIVABILITY RISK — ACT ON THIS: Backups older than 90 days are deleted automatically, and they live only inside GitHub. If GitHub access is lost, the backups are lost too. **At least monthly, download the latest backup artifact and store a copy somewhere independent** (association Google Drive / cloud storage). Also confirm Supabase's own Point-in-Time Recovery / daily backups are enabled on your plan as a second line of defence.

Run a backup right now (manually)

- GitHub → repository → **Actions** tab → **Automated Database Backup** → **Run workflow**.
- When it finishes (green tick), open the run and download the **.tar.gz** artifact from the bottom of the page.

Restore from a backup (worst case)

- Download and unzip the latest backup artifact. It contains `roles.sql`, `schema.sql`, and `data.sql`.
- In a clean/new Supabase project, run them via the SQL Editor or the Supabase CLI in this order: `roles` → `schema` → `data`.
- Update **VITE_SUPABASE_URL** and **VITE_SUPABASE_ANON_KEY** in Vercel to point at the restored project, then redeploy.
- Practise this once on a throwaway Supabase project before you ever need it for real.

7. Common problems — if you see X, do Y

Symptom	First thing to try
A scorer's balls aren't showing up for spectators	They are likely signed out / session expired. Have them sign in again; queued balls sync automatically. Confirm the device is online.
App shows old/blank screen after a deploy	Hard-refresh (Ctrl+F5) or close and reopen the app. It auto-updates; a stuck user just needs a reload.
Live scores not updating for viewers	Real-time reconnects on tab focus. Ask viewers to switch away and back, or reload. Check Supabase project is not paused.
Match marked complete but a correction is needed	Completed matches are locked by design (data integrity). A <code>SUPER_ADMIN</code> must make corrections; do not force edits in the DB without a backup.
Nobody can log in at all	Check Supabase project status (not paused/over quota) and that Vercel env vars still hold the correct Supabase URL/key.
Images won't upload	Cloudinary issue only — check the Cloudinary account and the upload preset. Scoring is unaffected.
Supabase emails 'project will pause'	The daily keep-alive ping may have failed. Re-run the keep-alive Action and check its secrets are set.

Where to look for the real error

- **Sentry** (if enabled) is the main window into production errors — the live site strips console logs, so Sentry is how you see what actually failed.
- **Supabase** → **Logs** shows database and auth errors.
- **Vercel** → **Deployments** → **Logs** shows build/deploy failures.

8. Emergency quick card

I need to...	Do this
Revert a broken release	Vercel → Deployments → promote last good one to Production
Back up the database now	GitHub → Actions → Automated Database Backup → Run workflow
Give someone scoring access	Supabase → profiles table → set role = SCORER
Lock someone out	Supabase → profiles table → set is_active = false
See what's broken	Sentry (errors) · Supabase Logs · Vercel Logs
Stop Supabase pausing	GitHub → Actions → Keep Supabase Alive → Run workflow

Keep this document current. Whenever an account owner, password location, or procedure changes, update it and re-issue. A runbook is only as good as its last revision.